

SQL & NoSQL

1. SQL

SQL (Structured Query Language) - Ngôn ngữ truy vấn có cấu trúc, là một ngôn ngữ lập trình chuyên biệt dùng để **tương tác** với **cơ sở dữ liệu**, cụ thể là:

- Thêm dữ liệu (INSERT),
- Sửa dữ liệu (UPDATE),
- Xóa dữ liệu (DELETE),
- Truy vấn dữ liệu (SELECT),
- Tạo/sửa cấu trúc bảng, chỉ mục, quyền truy cập (CREATE, ALTER, GRANT,...).

Đây là ngôn ngữ chính dùng trong các hệ **Cơ sở dữ liệu quan hệ (Relational Database Management System - RDBMS)** như: **MySQL, PostgreSQL, SQL Server, Oracle,...**

Structured data (dữ liệu có cấu trúc) là loại dữ liệu được **tổ chức rõ ràng theo hàng và cột** — giống như bảng Excel.

- Mỗi dòng (row) là một bản ghi (record),
- Mỗi cột (column) là một thuộc tính (field),
- Thường dùng **ký tự chữ số (alphanumeric)**.

Ví dụ: Dữ liệu giao dịch, danh sách khách hàng, hàng tồn kho,... đều là dữ liệu có cấu trúc.

Khi nói "**SQL database**", người ta đang nói đến **một cơ sở dữ liệu sử dụng SQL làm ngôn ngữ chính** để:

- Tạo bảng và lược đồ (schema),
- Thực hiện truy vấn (query),
- Quản lý người dùng và quyền truy cập,
- Giao tiếp thông qua các **API SQL**, giúp lập trình viên làm việc với dữ liệu dễ dàng mà không cần viết lại truy vấn mỗi lần.

Relational Database (Cơ sở dữ liệu quan hệ)

- Dữ liệu được **lưu trong bảng (table)** gồm **hàng (rows)** và **cột (columns)**.
- Các bảng có thể **liên kết với nhau** thông qua **khóa ngoại (foreign key)** – đây là cách thiết lập **mối quan hệ logic giữa các bảng**.

Ví dụ:

- Bảng KháchHang chứa thông tin khách hàng.
- Bảng DonHang chứa các đơn đặt hàng.
- DonHang có cột MaKhachHang là khóa ngoại liên kết với KháchHang.

Note: Relational databases are created and managed using **a fixed schema**.

Fixed schema - Schema cố định là gì?

Trong cơ sở dữ liệu quan hệ (SQL), **schema** là **cấu trúc định nghĩa trước** về:

- Tên bảng,
- Số lượng và loại cột,
- Kiểu dữ liệu của mỗi cột (int, varchar, date,...),
- Ràng buộc (constraint) như khóa chính, khóa ngoại, NOT NULL,...

Schema cố định (fixed schema) nghĩa là:

- **Dữ liệu đưa vào bắt buộc phải tuân theo đúng cấu trúc định nghĩa sẵn.**
- Nếu sai định dạng, thiếu trường, hoặc dữ liệu không đúng kiểu → **dữ liệu sẽ bị từ chối**.

Hạn chế của cơ sở dữ liệu quan hệ

- **Không phù hợp với dữ liệu phi cấu trúc:** Ví dụ như hình ảnh, văn bản tự do, file log, email, nội dung mạng xã hội,...
- **Khó thay đổi schema:** Khi muốn thêm cột mới hoặc thay đổi kiểu dữ liệu, cần thao tác ALTER TABLE → tốn thời gian và dễ ảnh hưởng dữ liệu cũ.
- **Thiếu linh hoạt với dữ liệu thay đổi cấu trúc liên tục** (ví dụ: thông tin người dùng từ nhiều nguồn khác nhau).

Tuy có hạn chế, **RDBMS vẫn rất mạnh** khi dùng với:

- **Dữ liệu có cấu trúc rõ ràng, ổn định** (structured/semi-structured).
- **Giao dịch tài chính, quản lý đơn hàng, hệ thống ERP,...** nhờ:
 - Tính nhất quán cao (ACID),
 - Hỗ trợ kiểm soát ràng buộc dữ liệu chặt chẽ.

There are a variety of different SQL database examples

Oracle	Hệ CSDL (RDBMS) mạnh cho doanh nghiệp, nổi tiếng về bảo mật, khả năng mở rộng.
MySQL	Mã nguồn mở, dễ dùng, phổ biến trong các ứng dụng web.
MariaDB	Bản fork mã nguồn mở từ MySQL, do cộng đồng phát triển.
PostgreSQL	Mạnh về tính năng, hỗ trợ dữ liệu phức tạp và tuân thủ ACID.
MSSQL (SQL Server)	Do Microsoft phát triển, dùng nhiều trong doanh nghiệp.
SQLite	Nhẹ, nhúng trong ứng dụng, không cần server riêng.

It's important to note that other types of databases can also **establish relationships between pieces of data**. In the case of normalized tabular databases (e.g., SQL or relational databases), these relationships are expressed using **foreign keys or intersection tables**.

- **Intersection Table (Bảng trung gian):** là một bảng đặc biệt dùng để biểu diễn mối quan hệ nhiều-nhiều (many-to-many) giữa hai bảng.
- Trong SQL, **không thể tạo trực tiếp mối quan hệ nhiều-nhiều** giữa hai bảng mà không thông qua một bảng trung gian.
- **Ví dụ đơn giản:** Giả sử bạn có hệ thống quản lý khóa học, với hai bảng:

Students (Sinh viên)			Courses (Khóa học)	
StudentID	Name		CourseID	Title
1	Alice		101	Database
2	Bob		102	Web Dev
Student_Courses (Intersection Table) - Một sinh viên có thể học nhiều khóa học, và một khóa học có thể có nhiều sinh viên. Đây là quan hệ nhiều-nhiều → Giải pháp: Tạo bảng Intersection - Mỗi dòng biểu diễn một quan hệ cụ thể giữa sinh viên và khóa học.			Student_Courses (Intersection Table)	
			StudentID	CourseID
			1	101
			1	102
			2	101

In the case of database management systems (DBMSs) such as MongoDB (e.g., a NoSQL database), these relationships are established by **embedding or referencing data**.

- **Embedding (Lồng dữ liệu)**

- Chèn toàn bộ document con vào trong document cha.
- Dùng khi dữ liệu con **thường truy xuất cùng dữ liệu cha** (1-nhiều, truy xuất nhanh).

```
{
  "name": "Alice",
  "orders": [
    { "id": 1, "item": "Laptop" },
    { "id": 2, "item": "Mouse" }
  ]
}
```

- **Referencing (Tham chiếu)**

- Lưu ID của document khác thay vì nhúng trực tiếp.
- Dùng khi dữ liệu con **được chia sẻ**, hoặc **kích thước lớn**, hoặc **truy xuất riêng biệt**.

```
{
  "name": "Alice",
  "order_ids": [101, 102]
}
```

→ Sau đó truy vấn Orders với `_id = 101` hoặc `102`.

2. NoSQL

NoSQL (viết tắt của “**Not Only SQL**”) là một nhóm các **hệ quản trị cơ sở dữ liệu không sử dụng mô hình quan hệ** (tức là không dùng bảng, cột như SQL truyền thống), và thường **không yêu cầu định nghĩa schema cố định**, giúp xử lý **dữ liệu lớn**, **linh hoạt**, **hiệu năng cao**.

Là một **hướng tiếp cận trong quản lý cơ sở dữ liệu**, cho phép:

- Lưu trữ và truy xuất **dữ liệu phi cấu trúc hoặc bán cấu trúc**,
- Không yêu cầu dữ liệu phải theo bảng hay **schema cố định** như trong SQL,
- Hỗ trợ các mô hình lưu trữ **phi quan hệ** (non-relational), ví dụ: document, key-value, column, graph.

Tên gọi “Not Only SQL” nhằm nhấn mạnh rằng NoSQL **không thay thế hoàn toàn SQL**, mà **mở rộng khả năng lưu trữ các dạng dữ liệu mà SQL không xử lý tốt**.

Khi người ta nói về NoSQL database, họ **không nói đến một ngôn ngữ**, mà họ thường ám chỉ các cơ sở dữ liệu **phi quan hệ (non-relational)**, tức là không sử dụng cấu trúc bảng cố định như các cơ sở dữ liệu truyền thống (SQL).

Một số người hiểu NoSQL là viết tắt của “**non-SQL**”, trong khi những người khác cho rằng nó là “**not only SQL**”. Dù hiểu theo cách nào, mọi người đều đồng ý rằng NoSQL lưu trữ dữ liệu một cách tự nhiên và linh hoạt hơn so với SQL.

Khác biệt chính là:

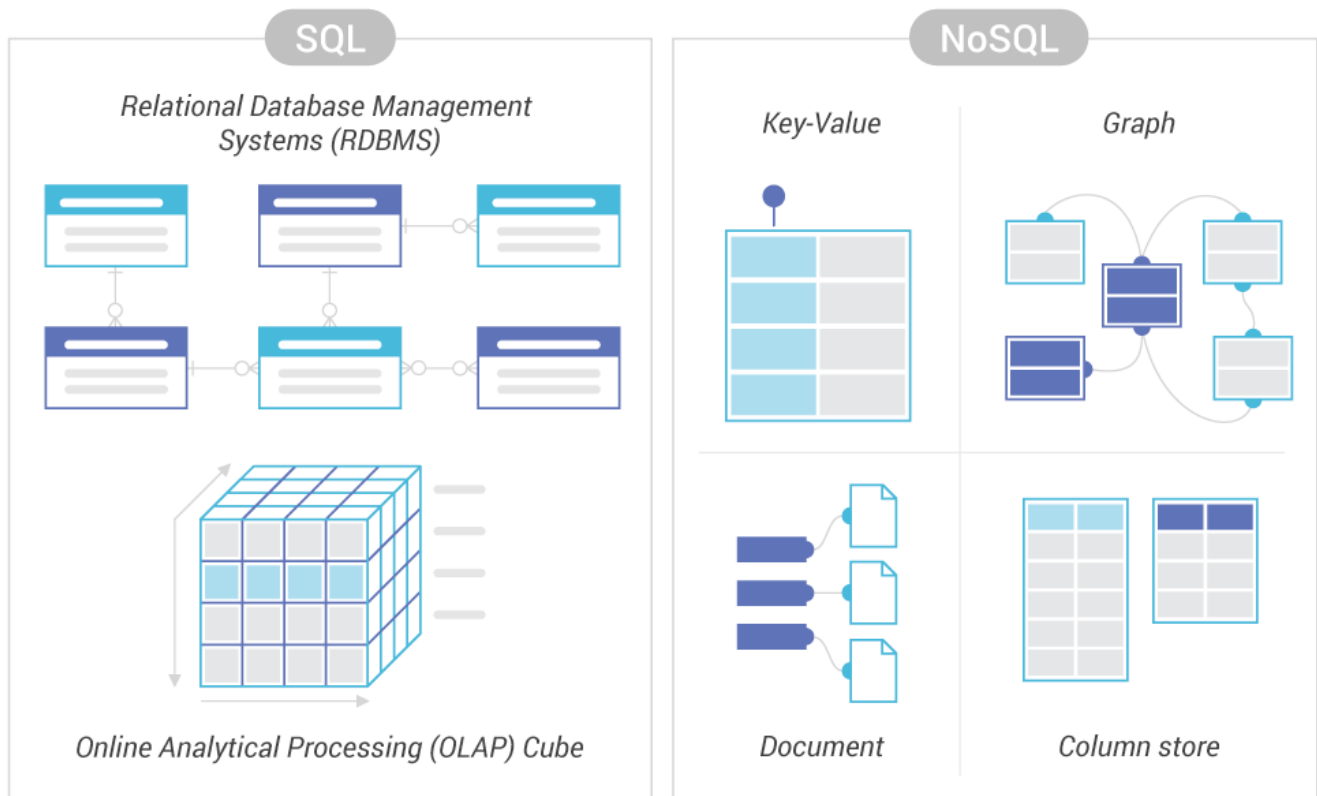
- **NoSQL** là một **cách tiếp cận quản lý cơ sở dữ liệu**, cho phép lưu trữ dữ liệu theo nhiều định dạng khác nhau (như key-value, document, graph, v.v.) mà không cần cấu trúc cố định.
- **SQL** chỉ là một **ngôn ngữ truy vấn** (query language) dùng để làm việc với dữ liệu trong các cơ sở dữ liệu quan hệ. Các cơ sở dữ liệu NoSQL cũng có ngôn ngữ truy vấn riêng, nhưng chúng linh hoạt hơn để phù hợp với dữ liệu đa dạng.

Nói đơn giản, NoSQL giống như một **"ngôi nhà" có thể chứa nhiều loại đồ vật khác nhau** mà không cần sắp xếp chúng theo cách cố định, còn **SQL giống như một "công cụ" để tìm và sắp xếp đồ** trong một ngôi nhà được tổ chức theo bảng rõ ràng.

Đặc điểm	Mô tả
Không dùng bảng-quan hệ (relation)	- sử dụng các mô hình lưu trữ đa dạng hơn. - Dữ liệu có thể được lưu dưới dạng document (tập hợp các cặp khóa-giá trị lồng nhau, thường là JSON hoặc BSON), key-value (cặp khóa duy nhất ánh xạ tới một giá trị bất kỳ), graph (các nút và cạnh thể hiện mối quan hệ), - hoặc column-family (lưu trữ theo cột, tối ưu cho các truy vấn theo hàng).
Không cần schema cứng	- không cần phải định nghĩa trước cấu trúc (schema) cho dữ liệu → Giúp giảm thời gian phát triển. - Có thể thêm/xóa các trường dữ liệu dễ dàng mà không cần sửa toàn bộ cấu trúc. - Lý tưởng để lưu trữ các loại dữ liệu như JSON, XML, tài liệu, hình ảnh, video, log files. - Điều này có thể dẫn đến dữ liệu không nhất quán nếu không được quản lý cẩn thận ở tầng ứng dụng.
Mở rộng dễ (horizontal scaling)	- Rất phù hợp với các ứng dụng có lượng dữ liệu khổng lồ và lưu lượng truy cập cao. Dễ dàng mở rộng hệ thống bằng cách thêm các "node" (máy chủ) mới vào một cụm. - Dữ liệu thường được phân tán (sharding) trên nhiều node, giúp xử lý lượng dữ liệu khổng lồ và lưu lượng truy cập cao một cách hiệu quả → Nếu một nút bị lỗi, các nút khác vẫn có thể hoạt động, đảm bảo tính liên tục của dịch vụ. - Điều này trái ngược với cơ sở dữ liệu SQL truyền thống thường mở rộng theo chiều dọc (nâng cấp một máy chủ mạnh hơn), vốn có giới hạn và tốn kém hơn.
Hiệu năng cao	- Tối ưu cho đọc/ghi nhanh, đặc biệt với dữ liệu lớn, truy cập thường xuyên. Bởi vì chúng không phải thực hiện các phép nối (JOIN) phức tạp giữa các bảng hoặc duy trì các ràng buộc toàn vẹn dữ liệu chặt chẽ như SQL. - NoSQL có thể cung cấp tốc độ truy cập dữ liệu vượt trội cho các trường hợp sử dụng phù hợp.
API đơn giản hơn	NoSQL thường cung cấp các API đơn giản hơn so với SQL, thường là các thao tác CRUD (Create, Read, Update, Delete) cơ bản dựa trên khóa hoặc ID tài liệu.
Các mô hình dữ liệu đa dạng	- Cho phép bạn chọn mô hình dữ liệu phù hợp nhất với nhu cầu ứng dụng của mình, dẫn đến hiệu suất và hiệu quả cao hơn. - Khả năng xử lý nhiều loại dữ liệu và mối quan hệ khác nhau.

	- Hầu hết các hệ thống NoSQL ưu tiên tính khả dụng (Availability) và khả năng chịu lỗi phân vùng (Partition Tolerance) hơn là tính nhất quán dữ liệu tuyệt đối (Consistency) theo mô hình ACID (Atomicity, Consistency, Isolation, Durability) truyền thống. - Điều này có nghĩa là trong trường hợp có sự cố mạng hoặc một số máy chủ bị lỗi, hệ thống vẫn có thể tiếp tục hoạt động và phục vụ yêu cầu (tính khả dụng cao), nhưng dữ liệu có thể không hoàn toàn nhất quán trên tất cả các node ngay lập tức (thường đạt được nhất quán cuối cùng - Eventual Consistency). - Điều này phù hợp cho các ứng dụng mà việc có dữ liệu hơi cũ một chút vẫn chấp nhận được, miễn là hệ thống luôn sẵn sàng. → NoSQL databases, như MongoDB hay Cassandra, thường được thiết kế để xử lý dữ liệu lớn và phân tán, nên chúng thường áp dụng mô hình BASE.
Không tuân thủ chặt ACID, mà tuân theo BASE và định lý CAP	ACID là tiêu chuẩn vàng (gold standard) cho các cơ sở dữ liệu quan hệ (RDBMS) như MySQL, PostgreSQL, hoặc Oracle. Nó đảm bảo tính chính xác và an toàn trong các giao dịch, đặc biệt khi xử lý dữ liệu quan trọng như ngân hàng hoặc thương mại điện tử. Tính nguyên tử (Atomicity): Hoặc là toàn bộ hoạt động được phản ánh đúng đắn (thực thi) trong CSDL, hoặc không có hoạt động nào cả (không có hoạt động nào được thực thi). Nếu xảy ra lỗi trong quá trình thực thi, tất cả các thao tác trong giao tác sẽ được hoàn tác về ban đầu (rollback). Ví dụ: Nếu bạn chuyển tiền, cả việc trừ tiền từ tài khoản A và cộng vào tài khoản B phải thành công, nếu không thì hủy toàn bộ. Tính nhất quán (Consistency): Khi một giao tác kết thúc (thành công hay thất bại), CSDL phải ở trạng thái nhất quán (Đảm bảo mọi RBTV). Một giao tác đưa CSDL từ trạng thái nhất quán này sang trạng thái nhất quán khác. Ví dụ: Nếu một ràng buộc rằng Balance ≥ 0 được thiết lập cho tài khoản ngân hàng, giao tác phải đảm bảo rằng không có tài khoản nào có số dư âm sau khi giao tác hoàn thành. Tính cô lập (Isolation): Một giao tác khi thực hiện sẽ không bị ảnh hưởng bởi các giao tác khác thực hiện đồng thời với nó. Ví dụ: Hai người cùng chuyển tiền từ cùng một tài khoản sẽ không bị lẫn lộn. Tính bền vững (Durability): Sau khi giao tác commit thành công, tất cả những thay đổi trên CSDL mà giao tác đã thực hiện phải được ghi nhận chắc chắn (vào ổ cứng). Mọi thay đổi trên CSDL được ghi nhận bền vững vào thiết bị lưu trữ dù có sự cố có thể xảy ra. SQL Server đảm bảo tính bền vững thông qua transaction log. Ví dụ: Khi bạn chạy một giao tác và COMMIT, thay đổi của nó sẽ được ghi vào log và lưu trên đĩa. Ngay cả khi hệ thống bị tắt đột ngột, cơ sở dữ liệu vẫn có thể khôi phục từ log. BASE là cách tiếp cận của nhiều hệ thống NoSQL (như MongoDB, Cassandra), tập trung vào tính sẵn sàng (availability) và hiệu suất, đặc biệt trong các hệ thống phân tán hoặc xử lý dữ liệu lớn. Basically Available (Cơ bản có sẵn): Hệ thống luôn cố gắng cung cấp dữ liệu, ngay cả khi một phần hệ thống gặp lỗi. Có thể có dữ liệu không đầy đủ trong trường hợp khẩn cấp.

	○ Soft State (Trạng thái mềm): Dữ liệu có thể thay đổi theo thời gian, không cần phải nhất quán ngay lập tức trên tất cả các nút trong hệ thống phân tán. ○ Eventually Consistent (Nhất quán cuối cùng): Dữ liệu sẽ trở nên nhất quán sau một thời gian, không cần phải đồng bộ ngay lập tức. Ví dụ: Khi bạn cập nhật thông tin trên mạng xã hội, có thể mất vài giây để hiển thị đồng bộ trên tất cả máy chủ. • Ứng dụng: Phù hợp với các hệ thống cần xử lý dữ liệu lớn, phân tán, như mạng xã hội, thương mại điện tử, hoặc ứng dụng thời gian thực.
	• CAP được đề xuất bởi nhà khoa học máy tính Eric Brewer. Đây là một định lý (Brewer's Theorem) cho rằng trong một hệ thống phân tán, bạn chỉ có thể đảm bảo tối đa hai trong ba đặc điểm C, A, P cùng lúc, không thể đạt cả ba hoàn toàn. • 3 thành phần của CAP: ○ Consistency (Nhất quán): Tất cả các nút (máy chủ) trong hệ thống phải có cùng một dữ liệu tại cùng một thời điểm. Nếu bạn cập nhật dữ liệu ở một nút, các nút khác phải thấy ngay thay đổi đó. Ví dụ: Khi bạn chuyển tiền, số dư tài khoản phải được cập nhật đồng bộ trên mọi máy chủ. ○ Availability (Khả dụng): Hệ thống luôn phản hồi yêu cầu của người dùng, ngay cả khi một số nút gặp sự cố. Dù có lỗi, hệ thống vẫn hoạt động và cung cấp dữ liệu (có thể không phải là dữ liệu mới nhất). Ví dụ: Một trang web thương mại điện tử vẫn cho phép đặt hàng dù một máy chủ bị lỗi. ○ Partition Tolerance (Dung sai phân vùng): Hệ thống vẫn hoạt động bình thường ngay cả khi có sự cố mạng chia cắt (partition) giữa các nút. Điều này rất phổ biến trong hệ thống phân tán khi mạng không ổn định. Ví dụ: Nếu hai trung tâm dữ liệu mất kết nối, hệ thống vẫn tiếp tục chạy ở cả hai bên. • Định lý CAP: ○ Bạn không thể có cả Consistency, Availability, và Partition Tolerance cùng lúc trong mọi tình huống. ○ Khi xảy ra phân vùng (partition), bạn phải chọn giữa: ▪ CA (Consistency + Availability): Đảm bảo nhất quán và khả dụng, nhưng không dung sai phân vùng (hệ thống có thể ngừng hoạt động nếu mạng bị chia cắt). ▪ CP (Consistency + Partition Tolerance): Đảm bảo nhất quán và dung sai phân vùng, nhưng có thể giảm khả dụng (hệ thống có thể từ chối yêu cầu nếu dữ liệu không đồng bộ). ▪ AP (Availability + Partition Tolerance): Đảm bảo khả dụng và dung sai phân vùng, nhưng chấp nhận dữ liệu không nhất quán ngay lập tức (sẽ nhất quán sau, giống BASE) → NoSQL thường chọn AP.
	• CAP là nền tảng để hiểu cách NoSQL hoạt động trong hệ thống phân tán, và BASE là cách NoSQL đạt được nhất quán trong môi trường này.



Dữ liệu phi cấu trúc (Unstructured Data) là gì?

- Là dữ liệu **không có cấu trúc cố định** hoặc không được tổ chức theo một mô hình cụ thể (như bảng trong cơ sở dữ liệu quan hệ/ không theo hàng, cột). Nó thường ở dạng tự do và không tuân theo một schema (lược đồ) được định nghĩa trước.
- Thay đổi linh hoạt, không đồng nhất.
- Thường chiếm khối lượng lớn và đa dạng về loại dữ liệu.
- Khó phân tích bằng SQL truyền thống vì không có định dạng chuẩn.
- Không có metadata hay key-value rõ ràng.
- Thường cần kỹ thuật xử lý ngôn ngữ tự nhiên (NLP), xử lý ảnh, audio analysis...
- **Ứng dụng:**
 - Phân tích dữ liệu lớn (Big Data): như phân tích cảm xúc từ bài đăng mạng xã hội.
 - Lưu trữ đa phương tiện: quản lý video, ảnh trên các nền tảng như YouTube, Instagram.
 - NoSQL databases (như MongoDB, Cassandra) thường được dùng để xử lý dữ liệu phi cấu trúc vì tính linh hoạt.

Dữ liệu bán cấu trúc (semi-structured data)

- Là loại dữ liệu **không tuân theo cấu trúc bảng cứng nhắc** như trong SQL (hàng, cột). Nó thường chứa các thẻ (tags) hoặc cấu trúc lỏng lẻo (như JSON, XML) để tổ chức dữ liệu, giúp dễ dàng truy vấn hơn so với dữ liệu phi cấu trúc.
- Mỗi bản ghi có thể có **schema khác nhau**, hoặc **thiếu trường**, mà hệ thống vẫn đọc được.
- Linh hoạt, dễ thay đổi cấu trúc mà không cần sửa đổi toàn bộ cơ sở dữ liệu.
- Có thể thêm/xóa trường mà không ảnh hưởng toàn hệ thống.
- Dễ truy vấn hơn dữ liệu phi cấu trúc, nhưng phức tạp hơn dữ liệu có cấu trúc (structured data).

- **Ứng dụng:**
 - Lưu trữ thông tin người dùng trong ứng dụng web hoặc mobile.
 - Quản lý cấu hình hệ thống hoặc dữ liệu từ API.
 - NoSQL databases (như MongoDB, CouchDB) rất phù hợp để xử lý dữ liệu bán cấu trúc vì chúng hỗ trợ các định dạng như JSON hoặc BSON.

Ví dụ:

Dữ liệu có cấu trúc (Structured)	Dữ liệu phi cấu trúc (Unstructured)	Dữ liệu bán cấu trúc (semi-structured data)
+ Giao dịch ngân hàng + Bảng tính Excel + Danh sách khách hàng	+ Ảnh chụp X-ray + Bài đăng mạng xã hội ("Hôm nay trời đẹp, mình đi dạo công viên!" kèm theo một bức ảnh) + Tập âm thanh MP3, video, bản đồ GPS + Văn bản tự do (email, bài đăng mạng xã hội, báo cáo văn bản) + Ảnh, video, âm thanh, PDF, PowerPoint...	File JSON <pre>{ "name": "Nguyen Van A", "age": 25, "hobbies": ["reading", "traveling"] }</pre> XML <pre><user> <name>Alice</name> <age>25</age> <skills> <skill>Java</skill> <skill>MongoDB</skill> </skills> </user></pre> File Log (key=value) <pre>timestamp=2025-07-29T20:11:00Z level=INFO user=alice action=login</pre>

NoSQL database là gì?

Là loại cơ sở dữ liệu:

- **Dùng schema linh hoạt** (không cần định nghĩa trước cột, kiểu dữ liệu),
- **Hỗ trợ lưu trữ dữ liệu không theo bảng**, ví dụ: File .txt, .jpg, .mp3 → vẫn có thể được lưu và truy xuất.
- **Một "collection" trong NoSQL có thể chứa nhiều bản ghi khác nhau về cấu trúc.**

So sánh nhỏ:

- **SQL:** Tất cả bản ghi trong 1 bảng **phải cùng định dạng**.
- **NoSQL:** Các bản ghi trong 1 collection **có thể khác định dạng hoàn toàn**.

Ví dụ thực tế:

Giả sử bạn xây dựng một ứng dụng quản lý bệnh án:

- Dữ liệu bệnh nhân gồm: thông tin cá nhân, kết quả xét nghiệm, ảnh chụp X-quang, file âm thanh bác sĩ ghi chú,...
- Nếu dùng SQL: Phức tạp, vì ảnh và file phải chuyển đổi sang kiểu BLOB, schema cứng nhắc.
- Nếu dùng NoSQL (như MongoDB): Có thể lưu ảnh, văn bản, âm thanh... **trực tiếp**, với mỗi bệnh án là một document riêng biệt và linh hoạt.

Hạn chế chung của NoSQL (so với SQL):

- **Không có chuẩn hóa (Standardization):** Không có ngôn ngữ truy vấn tiêu chuẩn như SQL, mỗi hệ thống NoSQL có ngôn ngữ truy vấn riêng.
- **Thiếu ràng buộc toàn vẹn:** Ít hỗ trợ các ràng buộc khóa ngoại (foreign key constraints), ràng buộc duy nhất (unique constraints) hoặc các ràng buộc toàn vẹn khác mà SQL cung cấp.
- **Hạn chế các phép nối phức tạp:** Các phép nối (JOIN) dữ liệu từ nhiều "bảng" hoặc tài liệu có thể không được hỗ trợ hoặc hiệu quả.
- **Hỗ trợ giao dịch (Transactions):** Mặc dù một số hệ thống NoSQL đã cải thiện hỗ trợ giao dịch, nhưng chúng thường không mạnh mẽ hoặc toàn diện như giao dịch ACID trong SQL.

3. Types of NoSQL databases

Có 4 loại chính:

3.1 Cơ sở dữ liệu định hướng tài liệu (Document-oriented databases)

- Cơ sở dữ liệu này lưu trữ dữ liệu dưới dạng **tài liệu**, giống như các đối tượng JSON (JavaScript Object Notation).
- Đôi khi được gọi là cơ sở dữ liệu hướng đối tượng (object-oriented databases).
- Each document itself is treated as a record.
- Mỗi tài liệu chứa các cặp trường (field) và giá trị (value).
- Giá trị có thể là nhiều loại khác nhau như chuỗi (string), số (number), boolean (đúng/sai), mảng (array). In addition, key-value pairs, nested documents, or other structured data can be included.
- Loại cơ sở dữ liệu này rất linh hoạt, phù hợp với dữ liệu bán cấu trúc (semi-structured) hoặc không cấu trúc (unstructured).
- Nó cũng hỗ trợ cấu trúc lồng nhau (nested structures), giúp dễ dàng biểu diễn các mối quan hệ phức tạp hoặc dữ liệu phân cấp.
- **Ví dụ:** MongoDB và Couchbase.

```
{
  "_id": "12345",
  "name": "foo bar",
  "email": "foo@bar.com",
  "address": {
    "street": "123 foo street",
    "city": "some city",
    "state": "some state",
    "zip": "123456"
  },
  "hobbies": ["music", "guitar", "reading"]
}
```

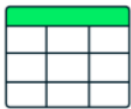
Dữ liệu được tổ chức thành các "tài liệu" riêng lẻ, mỗi tài liệu có thể khác nhau về cấu trúc, rất tiện lợi cho các ứng dụng như quản lý nội dung hoặc dữ liệu người dùng.

3.2. Cơ sở dữ liệu key-value (Key-value databases)

- Đây là loại cơ sở dữ liệu đơn giản, nơi mỗi mục chứa **một khóa (key)** duy nhất và **một giá trị (value)** đi kèm.
- The key is used to retrieve the corresponding value from the database. For example, in an interior design key-value database, a key might be "color" and the value might be "purple."

- Chúng thường được dùng để **lưu trữ tạm (caching)** hoặc quản lý phiên (session management), và có hiệu suất cao trong việc đọc/ghi vì thường lưu dữ liệu trong bộ nhớ (in-memory).
- Ví dụ:** Amazon DynamoDB và Redis.
 Key: user:12345
 Value: {"name": "foo bar", "email": "foo@bar.com", "designation": "software developer"}
- Hình dung như một tủ khóa: mỗi khóa là tên ngăn kéo, và bên trong là thông tin (giá trị). Loại này rất nhanh nhưng chỉ phù hợp với dữ liệu đơn giản.

RDBMS vs NoSQL (Document)



Relational Database



MongoDB

User table

ID	first_name	last_name	cell	city
1	Leslie	Yepp	8125552344	Pawnee

Hobbies table

ID	user_id	hobby
10	1	scrapbooking
11	1	eating waffles
12	1	working

```
{
  "_id": 1,
  "first_name": "Leslie",
  "last_name": "Yepp",
  "cell": "8125552344",
  "city": "Pawnee",
  "hobbies": ["scrapbooking", "eating waffles", "working"]
}
```

- No need for joins
- No need for data normalization

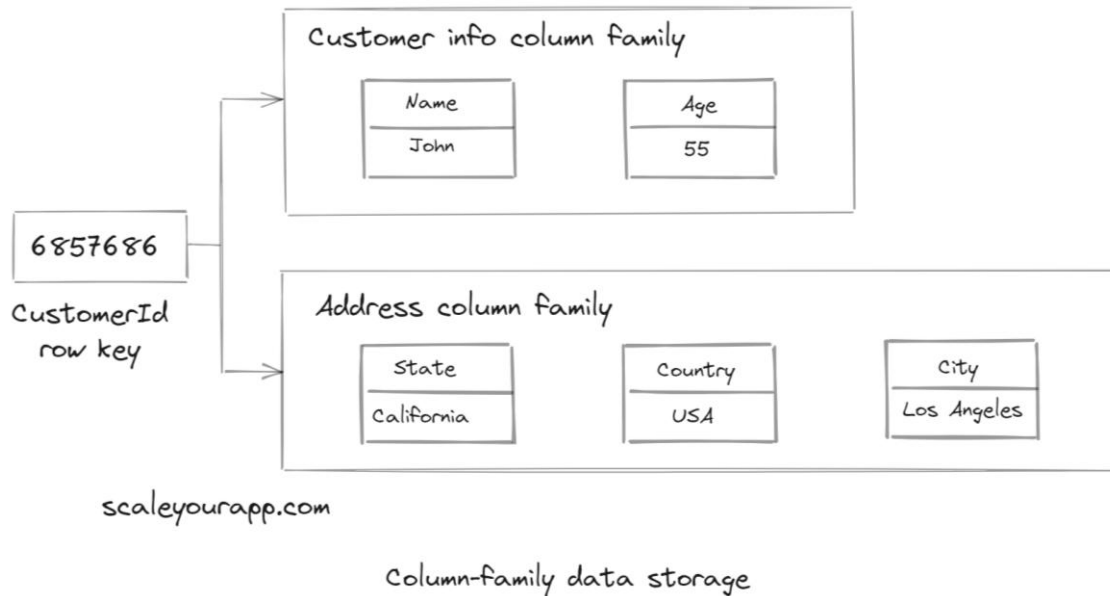
3.3. Cơ sở dữ liệu cột rộng (Wide-column stores)

- In fact, **column-family stores** are sometimes referred to as "wide-column stores."
- Loại này lưu dữ liệu trong **bảng, hàng, và cột động**.
- Dữ liệu được tổ chức thành bảng, nhưng khác với cơ sở dữ liệu SQL truyền thống, **các hàng có thể có các cột khác nhau**.
- Lưu trữ và truy vấn **dữ liệu theo cột**, thay vì theo hàng như trong SQL truyền thống. Ví dụ: Khi bạn cần **tổng hợp hoặc thống kê một vài cột trên rất nhiều dòng** (ví dụ: tính trung bình tuổi của 10 triệu người).
- Each row has a different set of columns, **with columns then gathered into "families."**
- Chúng sử dụng kỹ thuật nén cột để tiết kiệm dung lượng và tăng hiệu suất.
- Các hàng và cột rộng giúp truy xuất dữ liệu thưa thớt (sparse) và rộng (wide) một cách hiệu quả.
- Ví dụ:** Apache Cassandra và HBase.
- Ví dụ về dữ liệu:**
 - Ví dụ a:

name	id	email	dob	city
Foo bar	12345	foo@bar.com		AB city
Carn Yale	34521	bar@foo.com	12-05-1972	

- **Foo bar và Carn Yale không cần có cùng cột.** Mỗi hàng là một bộ thông tin riêng, phù hợp cho dữ liệu lớn và thay đổi linh hoạt.

○ Ví dụ b:



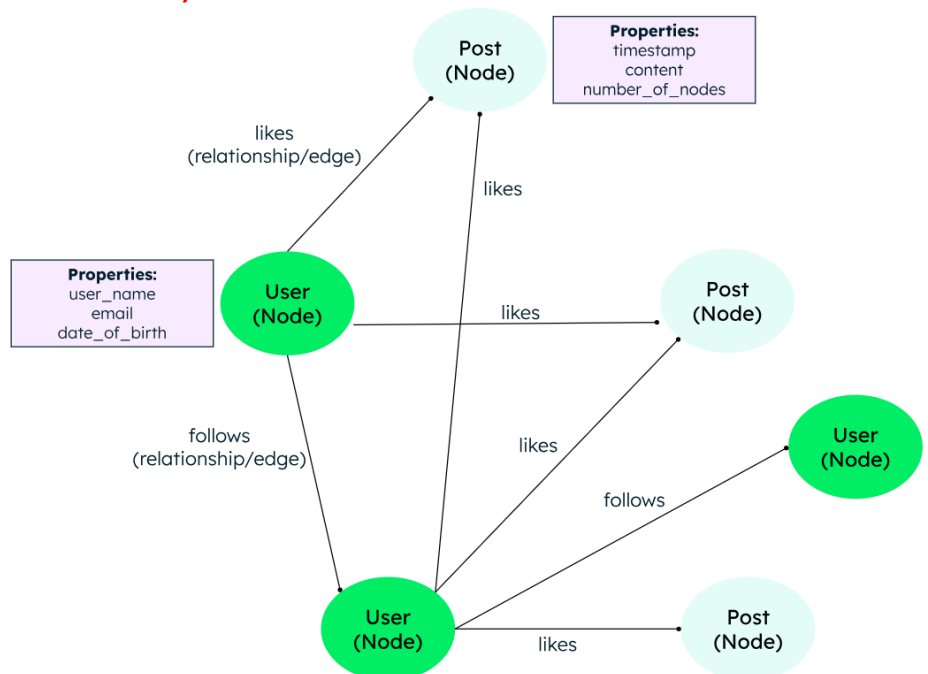
▪ Nếu chuyển thành dạng bảng:

CustomerId	Name	Age	State	Country	City
6857686	John	55	California	USA	Los Angeles

- Dữ liệu được chia nhỏ thành **các nhóm cột có liên quan**, ví dụ: "Customer info" và "Address".
- Mỗi nhóm này là một **column family**.
- Dữ liệu được truy cập theo **Row Key**, ở đây là CustomerId = 6857686.

3.4. Cơ sở dữ liệu đồ thị (Graph databases)

- Cơ sở dữ liệu này lưu dữ liệu dưới dạng **nút (nodes)** và **đường nối (edges)**.
- Nút thường chứa thông tin về người, địa điểm, hoặc vật (như danh từ), trong khi đường nối chứa thông tin về mối quan hệ giữa các nút.
- Chúng rất hiệu quả với dữ liệu có kết nối chặt chẽ, đặc biệt khi các mối quan hệ không rõ ràng ban đầu.
- Graph databases are excellent tools for querying graph structures (e.g., social networks, hierarchies).
- **Ví dụ:** Neo4J và Amazon Neptune. MongoDB cũng hỗ trợ khả năng duyệt đồ thị (graph traversal) bằng lệnh \$graphLookup trong pipeline tổng hợp.



3.5. Cơ sở dữ liệu đa mô hình (Multi-model databases)

- Đây là loại cơ sở dữ liệu hỗ trợ **nhiều mô hình NoSQL** trong cùng một hệ thống, cho phép nhà phát triển chọn mô hình phù hợp với nhu cầu ứng dụng.
- Chúng có một **động cơ thống nhất** để xử lý nhiều loại dữ liệu trong một instance cơ sở dữ liệu.
- **Ví dụ:** CosmosDB và ArangoDB.
- **Ví dụ dữ liệu:** Một ứng dụng có thể dùng mô hình tài liệu cho dữ liệu người dùng và mô hình đồ thị cho mối quan hệ giữa họ, tất cả trong cùng một cơ sở dữ liệu.

4. Khi nào cần dùng NoSQL?

NoSQL được thiết kế để giải quyết những hạn chế của hệ quản trị cơ sở dữ liệu quan hệ (SQL) trong một số trường hợp cụ thể. Bạn nên dùng **NoSQL** khi:

- **Dữ liệu phi cấu trúc hoặc bán cấu trúc hoặc thay đổi thường xuyên.** Ví dụ: Hồ sơ người dùng có thể khác nhau: người có địa chỉ, người không; người có nhiều số điện thoại, người không có; dữ liệu từ mạng xã hội, IoT, log hệ thống...
- **Quy mô lớn, yêu cầu mở rộng theo chiều ngang (horizontal scaling):** Khi bạn cần xử lý lượng lớn dữ liệu (Big Data) và yêu cầu mở rộng dễ dàng bằng cách thêm nhiều máy chủ hoặc nút vào hệ thống. SQL thường mở rộng theo chiều dọc (tăng RAM, CPU), còn NoSQL mở rộng tốt hơn theo chiều ngang (thêm node).
- **Hiệu suất cao, yêu cầu truy vấn siêu nhanh:** Khi yêu cầu tốc độ đọc/ghi rất cao, ví dụ như ứng dụng mạng xã hội, real-time chat, IoT,... NoSQL thường cho tốc độ cao hơn nhờ vào mô hình đơn giản và không cần JOIN phức tạp.
- **Dễ thay đổi schema, Mô hình dữ liệu linh hoạt:** NoSQL không yêu cầu định nghĩa schema cứng nhắc ban đầu. Bạn có thể thêm trường mới mà không cần ALTER TABLE, thích hợp với các ứng dụng Agile, MVP,... Khi cấu trúc dữ liệu thay đổi thường xuyên, NoSQL cho phép thay đổi schema mà không cần sửa đổi toàn bộ cơ sở dữ liệu.
- **Hệ thống phân tán (distributed system):** NoSQL được thiết kế để hoạt động tốt trong môi trường phân tán, ví dụ như Cassandra, MongoDB, Couchbase,...
- **Không yêu cầu tính ACID (Atomicity, Consistency, Isolation, Durability) nghiêm ngặt cho mọi giao dịch:** Mặc dù một số cơ sở dữ liệu NoSQL có thể cung cấp mức độ nhất quán nhất định, nhưng chúng thường ưu tiên tính sẵn sàng (Availability) và khả năng chịu lỗi (Partition tolerance) hơn là tính nhất quán mạnh mẽ của ACID, phù hợp cho các ứng dụng không cần đảm bảo toàn vẹn dữ liệu ở mức giao dịch tuyệt đối.
- **Phát triển nhanh (Rapid Development):** Do không yêu cầu lược đồ cố định, NoSQL cho phép các nhà phát triển chèn dữ liệu mà không cần xác định trước lược đồ, giúp đẩy nhanh quá trình phát triển và thay đổi ứng dụng.
- **Lưu trữ dữ liệu trên Cloud Computing:** NoSQL thường hoạt động tốt trên môi trường điện toán đám mây, tận dụng tối đa lợi thế về khả năng mở rộng và tiết kiệm chi phí.

Ví dụ ứng dụng thực tế:

- **Mạng xã hội:** Lưu trữ dữ liệu người dùng, bài đăng, tương tác, tin nhắn (dữ liệu phi cấu trúc và thay đổi liên tục).
- **IoT (Internet of Things):** Xử lý và lưu trữ dữ liệu cảm biến khổng lồ từ các thiết bị IoT.
- **Hệ thống phân tích dữ liệu lớn thời gian thực (Big Data Analytics):** Phân tích luồng dữ liệu lớn với hiệu suất cao.

- **Hệ thống quản lý nội dung (CMS) hoặc thương mại điện tử:** Khi cần lưu trữ nhiều loại dữ liệu khác nhau cho sản phẩm, bài viết, bình luận mà không bị ràng buộc bởi cấu trúc cứng nhắc.

5. Có thể kết hợp SQL và NoSQL không?

- Có thể kết hợp, và thực tế rất nhiều hệ thống hiện đại đang sử dụng kiến trúc kết hợp (**polyglot persistence**) – tức là dùng nhiều hệ quản trị dữ liệu khác nhau tùy theo nhu cầu.
- Kết hợp SQL và NoSQL là một chiến lược hiệu quả khi ứng dụng của bạn có các yêu cầu về dữ liệu đa dạng mà một loại cơ sở dữ liệu không thể đáp ứng tối ưu:
 - **Dữ liệu có tính chất khác nhau:**
 - **SQL (cơ sở dữ liệu quan hệ) cho dữ liệu có cấu trúc, quan trọng và yêu cầu tính ACID cao:** Ví dụ: **thông tin tài khoản ngân hàng, hồ sơ khách hàng, đơn hàng, dữ liệu kế toán**. Những dữ liệu này cần đảm bảo tính toàn vẹn, nhất quán và quan hệ chặt chẽ giữa các bảng.
 - **NoSQL (cơ sở dữ liệu phi quan hệ) cho dữ liệu phi cấu trúc/bán cấu trúc, khối lượng lớn và yêu cầu khả năng mở rộng cao:** Ví dụ: dữ liệu log, **phiên người dùng, dữ liệu cảm biến, bình luận, tin nhắn, dữ liệu phân tích, thông tin cá nhân mở rộng** (profile người dùng với nhiều trường tùy chọn).
 - **Tối ưu hóa hiệu suất và khả năng mở rộng:**
 - Có những phần của ứng dụng yêu cầu truy vấn quan hệ phức tạp, đảm bảo ACID (SQL).
 - Có những phần khác cần **tốc độ cao để ghi và đọc lượng lớn dữ liệu** mà không cần quan hệ phức tạp, và cần mở rộng dễ dàng (NoSQL).
 - **Mô hình thiết kế ứng dụng cụ thể:**
 - **CQRS (Command Query Responsibility Segregation):** Tách biệt các tác vụ ghi (command) và đọc (query). **Dữ liệu ghi có thể dùng SQL** để đảm bảo tính nhất quán, trong khi **dữ liệu đọc có thể được lưu trữ trên NoSQL** (ví dụ: Elasticsearch, MongoDB) để tối ưu hóa truy vấn và hiệu suất đọc.
 - **Event Sourcing:** Lưu trữ tất cả các sự kiện (events) xảy ra trong hệ thống. Các sự kiện này có thể được lưu trữ trong NoSQL (như Kafka, Cassandra) vì chúng thường là dữ liệu phi cấu trúc, chỉ cần ghi nhanh và có khả năng mở rộng. Trạng thái hiện tại của hệ thống có thể được xây dựng từ các sự kiện này và lưu vào SQL nếu cần các truy vấn phức tạp hoặc báo cáo.

- **Khi nào nên kết hợp?**

Tình huống	SQL	NoSQL	Mục đích kết hợp
Quản lý dữ liệu giao dịch (đơn hàng, thanh toán...)	✅ Được sử dụng để lưu trữ các giao dịch quan trọng.	❌ Ít phù hợp cho dữ liệu giao dịch phức tạp cần đảm bảo ACID.	SQL đảm bảo tính nhất quán (ACID), đảm bảo mọi giao dịch đều an toàn, toàn vẹn.
Lưu trữ log, cache, hoặc phiên người dùng	❌ Không tối ưu cho dữ liệu lớn, thay đổi nhanh.	✅ Được sử dụng rộng rãi để lưu trữ dữ liệu không cần cấu trúc chặt chẽ, tốc độ ghi/đọc cao.	SQL sẽ trở nên chậm chạp và kém hiệu quả khi phải ghi/đọc liên tục lượng lớn dữ liệu không cần tính toàn vẹn giao dịch.

<i>Hệ thống có phân phân tích (BI, dashboard)</i>	✅ Thường dùng để lưu trữ dữ liệu đã được xử lý, chuẩn hóa để phân tích, tạo báo cáo.	✅ Sử dụng để thu thập dữ liệu thô (raw data), dữ liệu phi cấu trúc, hoặc bán cấu trúc từ nhiều nguồn khác nhau.	SQL để phân tích, NoSQL để thu thập/tiền xử lý dữ liệu thô và phục vụ các truy vấn nhanh cho các dashboard tức thời. SQL cung cấp khả năng truy vấn phức tạp (JOIN, GROUP BY) tốt hơn cho BI truyền thống, trong khi NoSQL giúp xử lý Big Data và dữ liệu streaming.
<i>Website thương mại điện tử</i>	✅ Lưu trữ thông tin đơn hàng, thông tin khách hàng cốt lõi (tên, địa chỉ, lịch sử mua hàng chính thức), dữ liệu tồn kho sản phẩm (cần tính nhất quán cao).	✅ Lưu trữ lịch sử duyệt web, sản phẩm đã xem, giỏ hàng (thay đổi, không quan trọng ACID), đánh giá và bình luận sản phẩm, dữ liệu phiên (session data).	Tối ưu cho cả hiệu năng lẫn dữ liệu ổn định
<i>Một ứng dụng mạng xã hội (Facebook)</i>	✅ Lưu trữ thông tin tài khoản người dùng, quan hệ bạn bè (cần đảm bảo nhất quán và các truy vấn quan hệ phức tạp).	✅ Lưu trữ các bài đăng, bình luận, lượt thích, tin nhắn (dữ liệu phi cấu trúc, khối lượng lớn, cần mở rộng nhanh).	SQL đảm bảo tính toàn vẹn của hồ sơ người dùng và mối quan hệ xã hội. NoSQL cung cấp hiệu suất và khả năng mở rộng để xử lý dữ liệu tương tác.
<i>YouTube</i>	✅ Lưu thông tin video (tiêu đề, mô tả, quyền riêng tư, kênh), thông tin người dùng (tài khoản, cài đặt).	✅ Lưu lịch sử xem, gợi ý video, bình luận, lượt thích, dữ liệu phân tích tương tác người dùng (khối lượng lớn, thay đổi, tốc độ truy cập cao).	SQL quản lý các thông tin cấu trúc, ổn định của video và người dùng. NoSQL xử lý dữ liệu tương tác người dùng khổng lồ, thay đổi liên tục, giúp cung cấp các tính năng như gợi ý cá nhân hóa và quản lý nội dung động.

• Khi nào KHÔNG nên kết hợp?

- Hệ thống đơn giản, chỉ cần một loại dữ liệu và không có yêu cầu mở rộng cao. SQL có thể đủ, và việc thêm NoSQL sẽ làm tăng độ phức tạp không cần thiết.
- Nếu không cần mở rộng ngang hoặc xử lý dữ liệu phi cấu trúc, SQL thường là lựa chọn dễ quản lý hơn.
- Bạn chưa có đủ nguồn lực quản lý, vận hành nhiều loại cơ sở dữ liệu (mỗi loại có cách backup, monitor, query khác nhau).
- Đồng bộ hóa dữ liệu giữa hai hệ thống có thể phức tạp, đặc biệt khi cần đảm bảo tính nhất quán.
- Đội ngũ chưa quen với việc vận hành song song nhiều nền tảng dữ liệu.
- Vận hành cả SQL và NoSQL đòi hỏi tài nguyên lớn hơn (máy chủ, bảo trì, nhân sự), không phù hợp với các dự án nhỏ.